

Curve Ellittiche

Applebee Kelly

Maggio 2026

Chapter 1

Introduzione

La necessità di trasmettere informazioni riservate su canali di comunicazione pubblici ha reso la crittografia a chiave pubblica uno degli strumenti fondamentali dell'informatica moderna. Tra le costruzioni crittografiche asimmetriche, la crittografia su curve ellittiche (ECC) si distingue per l'elevata sicurezza ottenibile con chiavi di dimensioni ridotte, caratteristica che la rende particolarmente adatta ai sistemi con risorse computazionali limitate, tra cui i dispositivi IoT.

La cifratura a chiave pubblica è l'evoluzione della cifratura a chiave privata, quest'ultima infatti presentava una sola chiave, simmetrica, nota sia al mittente che al destinatario, comunicato senza utilizzare canali non protetti (di solito comunicati in persona evitando dispositivi connessi ad internet) in modo da poter successivamente instaurare una comunicazione su un canale sicuro, dove sia il mittente che il destinatario dispongono della stessa chiave.

La cifratura a chiave pubblica introduce una nuova chiave per ogni utente all'interno della comunicazione, una chiave pubblica usata per cifrare il messaggio, e una privata usata per decifrare il messaggio ricevuto; la chiave pubblica di cifrazione è nota a tutti gli utenti interessati a comunicare con una certa persona, la seconda chiave (di decifrazione) è privata e nota solamente ad un individuo, una volta ricevuto il messaggio cifrato con la chiave pubblica del destinatario, questo usa la sua chiave privata per decifrare il messaggio.

TO FIX ATTACK

L'attacco più comune compiuto contro sistemi a chiave pubblica si chiama *known-plaintext* consiste nel codificare una serie di messaggi con le informazioni date dal destinatario (la sua chiave pubblica e la funzione usata per cifrare i messaggi da inviare) per poi confrontare questi nuovi crittogrammi con quelli che vengono inviati su un canale di comunicazione, in modo da vedere se coincidono.

BROKEN BY QUANTUM COMPUTERS TO FIX

Alcuni degli algoritmi più popolari che utilizzano le chiavi pubbliche che rimangono al giorno d'oggi inviolati sono RSA e Diffie-Hellman;

1.1 Campi

Prima di parlare di algoritmi crittografici è necessario introdurre alcuni concetti matematici, primo tra i quali i *campi*.

Definizione 1. *Un campo $(K, +, \cdot)$ è un insieme non vuoto che presenta operazioni binarie le quali soddisfano le seguenti proprietà:*

- $(K, +)$ è un gruppo con elemento neutro 0
- $(K \setminus \{0\}, \cdot)$ è un gruppo abeliano con elemento neutro 1
- La moltiplicazione è distributiva rispetto all'addizione: $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$

In altre parole si vuole che ogni elemento, escluso lo 0 abbia un inverso moltiplicativo.

1.1.1 Esempi di campi

- L'insieme dei numeri razionali $\mathbb{Q}$ è un campo.
- L'insieme dei numeri reali $\mathbb{R}$ è un campo
- L'insieme $\mathbb{Z}/\mathbb{Z}_p$ delle classi di resto modulo p è un campo se e solo se p è un numero primo

1.2 Funzione di Eulero

Definizione 2. *La funzione di Eulero può essere scritta come $\Phi(n) = |\mathbb{Z}_n^*|$*

Questa funzione risulta utile perché dato:

- $\Phi(p) = p - 1$
- $\Phi(pq) = (p - 1)(q - 1)$
- a primo con b allora $\Phi(ab) = \Phi(a)\Phi(b)$
- $\Phi(n)$ difficile da calcolare senza conoscere la fattorizzazione di n

Definizione 3. Per ogni $p > 1$ primo e per ogni $k \geq 1$ intero:

$$\Phi(p^k) = \Phi(p) \cdot p^{k-1} \quad (1.1)$$

$$= (p - 1) \cdot p^{k-1} \quad (1.2)$$

1.3 RSA

Il cifrario RSA fonda la sua sicurezza sulla difficoltà di fattorizzazione di grandi numeri interi, il che lo rende sostanzialmente inviolabile per chiavi adeguatamente grandi

1.3.1 Creazione delle chiavi

Un Destinatario deve compiere queste operazioni in modo di creare le chiavi pubbliche e private:

1. scegliere due numeri primi molto grandi p, q
2. calcolare $n = p \cdot q$ e $\Phi(n) = (p - 1) * (q - 1)$
3. scegliere un numero intero e minore di $\Phi(n)$ e primo con esso
4. calcolare l'intero d inverso di e modulo $\Phi(n)$
5. rendere pubblica la chiave $k_{\text{pub}} = (e, n)$, e mantenere segreta la chiave $k_{\text{prv}} = d$

Per codificare correttamente il messaggio questo deve essere trasformato in binario, il quale viene trattato come un numero intero m , questo deve risultare $< n$, il che è possibile dividendo il messaggio m in blocchi di al più $\lfloor \log_2(n) \rfloor$ bit

$$c = m^e \bmod n \quad (\text{cifatura}) \quad (1.3)$$

$$m = c^d \bmod n \quad (\text{decifatura}) \quad (1.4)$$

1.4 Diffie-Hellman

A differenza di RSA Diffie-Hellman richiede una collaborazione da parte dei due utenti per la generazione delle chiavi:

1. L'utente A sceglie un numero primo p , un generatore g di $\mathbb{Z}_p^*$ e un intero $a > 0$

2. A calcola $A = g^a \bmod p$ e spedisce (g, p, A) a B
3. L'utente B sceglie un intero $b > 0$, calcola $B = g^b \bmod p$ e lo spedisce ad A
4. B calcola la chiave segreta $K = A^b \bmod p$
5. A calcola la chiave segreta $K = B^a \bmod p$

La sicurezza di questo protocollo è dato dal fatto che anche se un crittoanalista potrebbe aver intercettato i valori p , g , A , B scambiati in chiaro tra i due utenti, questo dovrebbe risolvere l'equazione $A = g^a \bmod p$ rispetto ad a oppure $B = g^b \bmod p$ rispetto a b ovvero calcolare il logaritmo discreto di A , o di B .

Chapter 2

Curve Ellittiche

2.1 Curve ellittiche su $\mathbb{R}$

Una curva ellittica è una curva piana definita da un'equazione del tipo:

$$y^2 = x^3 + ax + b^1 \quad (2.1)$$

Questa possiede un'operazione di somma fra punti appartenenti alla curva, rispetto alla quale essa è un gruppo abeliano il cui elemento neutro è costituito dal punto all'infinito O .

Una curva ellittica è caratterizzata dal fatto che sommando due punti se ne ottiene un terzo della stessa curva. Nella definizione di curva ellittica viene definito anche il *punto all'infinito* O definito come la somma di tre punti su una curva ellittica collegati da una retta.

2.1.1 Caratteristiche curve ellittiche

1. Chiusura:

$$\forall P, Q \in E(a, b) : P + Q \in E(a, b) \quad (2.2)$$

2. Elemento neutro:

$$\forall P \in E(a, b) : P + O = O + P = P \quad (2.3)$$

(vedere Somma su curve ellittiche)

3. Associatività:

$$\forall P, Q, R \in E(a, b) : P + (Q + R) = (P + Q) + R \quad (2.4)$$

¹Questo funziona solamente quando la caratteristica del campo K è diversa da 2 e 3, altrimenti l'equazione diventerebbe $y^2 + axy + by = x^3 + cx^2 + dx + e$ dove $a, b, c, d, e \in K$

4. Commutatività:

$$\forall P, Q \in E(a, b) : P + Q = Q + P \quad (2.5)$$

2.1.2 Punto all'infinito

Definizione 4. *Punto improprio definito in Geometria proiettiva associato a una direzione nel piano: ogni famiglia di rette parallele converge in un unico punto all'infinito, l'insieme di questi punti forma la retta impropria.*

Nelle curve ellittiche, il punto all'infinito $\mathcal{O}$ è unico ed funge da elemento neutro dell'addizione del gruppo.

Intuizione geometrica

I punti all'infinito corrispondono ai punti $[X : Y : Z]$ con $Z = 0$. Passando alle coordinate omogenee $[X : Y : Z]$ con $x = X/Z$, $y = Y/Z$, l'equazione di Weierstrass diventa:

$$Y^2Z = X^3 + aXZ^2 + bZ^3 \quad (2.6)$$

Ponendo $Z = 0$ si ottiene $X = 0$, da cui il punto all'infinito è $\mathcal{O} = [0 : 1 : 0]$ [3].

2.1.3 Somma su curve ellittiche

DA METTERE ELEMENTO NEUTRO IN SOMMA DI DUE PUNTI CON LA STESSA X

Definizione 5. *Ogni retta interseca una curva ellittica in al più tre punti [1]*

Dalla definizione sopra si può capire dedurre che se una retta interseca una curva $E(a, b)$ in due punti P e Q allora la retta interseca anche un terzo punto R nel caso $P \neq Q$; altrimenti la retta interseca soltanto due punti.

Se la retta è verticale invece, oltre ai punti P e Q interseca anche il punto all'infinito $\mathcal{O}$.

Dati tre punti P, Q, R di una curva ellittica $E(a, b)$ se P, Q e R sono disposti su una retta, allora la loro somma si pone uguale al punto all'infinito:

$$P + Q + R = \mathcal{O} \quad (2.7)$$

Da queste definizioni è possibile ricavare la regola per sommare due punti P e Q

- Si consideri la retta passante per i punti P e Q , oppure la tangente alla curva in P nel caso P e Q coincidano.
- Si determina il punto di intersezione R tra la curva e la retta appena tracciata
- Si definisce la somma come $P + Q = -R$ dove $-R$ rappresenta il punto simmetrico a R rispetto all'asse delle ascisse

Da un punto di vista pratico questo vuol dire:

Considerando P e Q distinti e $P \neq -Q$

1. $P = (x_p, y_p), Q = (x_q, y_q),$
2. $P + Q = S = (x_s, y_s)$
3. $\lambda = \frac{y_q - y_p}{x_q - x_p}$
4. $x_s = \lambda^2 - x_p - x_q$
5. $y_s = -y_p + \lambda(x_p - x_s)$

Se studiamo il caso in cui P e Q hanno la stessa coordinata x , perciò $Q = -P$, questi si trovano allineati lungo la verticale che interseca il punto all'infinito della curva.

Questa affermazione si può anche mostrare utilizzando la formula della somma di due punti $P + Q = P + (-P) = O$

Da un punto di vista matematico questo si vede nel seguente modo:

1. $P = (x_p, y_p), 2P = S = (x_s, y_s)$
2. $y_p \neq 0 \Rightarrow \lambda = \frac{(3x_p^2 + a)}{2y_p}$
3. $y_p = 0 \Rightarrow$ tangente verticale $\Rightarrow 2P = O$

2.1.4 Moltiplicazione scalare

La moltiplicazione scalare consiste nella ripetizione della somma di un punto a se stesso, un numero arbitrario grande k volte, segnato come $S = kP$. Il problema si incontra quando, k , inevitabilmente, diventa molto grande; per evitare la difficoltà computazionale richiesta per sommare migliaia di volte un punto su un'equazione si usa un algoritmo chiamato **double-and-add**

2.1.5 Double-and-add

L'algoritmo *double and add* si basa sui principi della scomposizione binaria e sulle proprietà delle curve ellittiche.

Per capire in quale modo consideriamo lo scalare k nella sua forma binaria:

$$k = (k_{n-1}k_{n-2} \dots k_1k_0)_2 \quad (2.8)$$

dove k_i sono le cifre binarie (bit) di k . La rappresentazione binaria ci consente di esprimere k come:

$$k = \sum_{i=0}^{n-1} k_i \cdot 2^i \quad (2.9)$$

Utilizzando la stessa rappresentazione si può rappresentare kP come:

$$kP = \left(\sum_{i=0}^{n-1} k_i \cdot 2^i \right) P \quad (2.10)$$

Per la proprietà distributiva questa espressione può essere scritta come:

$$kP = \sum_{i=0}^{n-1} k_i \cdot (2^i P) \quad (2.11)$$

Il termine $2^i P$ rappresenta il punto P raddoppiato i volte. Perciò il problema si riduce ad una serie di raddoppi e addizioni di punti, il che rappresenta l'essenza dell'algoritmo.

2.2 Curve su campo finito

Le curve ellittiche su $\mathbb{R}$ non sono di particolare interesse ai crittografici, si preferisce invece lavorare su curve ellittiche definite su campi finiti. Questa decisione permette di compiere operazioni veloci e precise dato dal fatto che tutte le operazioni riguardano coefficienti relativamente ridotti e interi, senza preoccuparsi di calcoli approssimati ed esageramente lunghi.

In aggiunta operare in aritmetica modulare ci permette di sviluppare problemi che risultano computazionalmente difficili.

Per il resto di questa relazione si utilizzerà un campo $\mathbb{Z}_p$ con caratteristica $p > 3$ in modo da utilizzare l'equazione ellittica ridotta alla forma normale di Weierstrass.

Definizione 6. Sia $p > 3$ un numero primo. La curva ellittica su $\mathbb{Z}_p$, denotata $E_p(a, b)$, è l'insieme dei punti che soddisfano l'equazione $y^2 = x^3 + ax + b \pmod{p}$ insieme al punto all'infinito.

$$E_p(a, b) = \{(x, y) \in \mathbb{Z}_p^2 : y^2 = x^3 + ax + b \pmod{p}\} \cup \{O\} \quad (2.12)$$

dove in aggiunta:

$$4a^3 + 27b^2 \neq 0 \quad (2.13)$$

Theorem 2.2.1 (Hasse). Sia N il numero di punti di una curva ellittica E definita su $\mathbb{F}_q$. Allora

$$|N - (q + 1)| \leq 2\sqrt{q} \quad (2.14)$$

Per la dimostrazione si veda [3]

I punti della curva $E_p(a, b)$ che rispettano queste due condizioni formano un gruppo abeliano finito rispetto all'operazione di addizione. [3]

2.2.1 Creazione di chiavi su $\mathbb{Z}_p$

Due utenti Angelica e Brian vogliono instaurare una comunicazione sicura, si concordano pubblicamente su un campo finito e su una curva ellittica definita su questo campo. Dopodiché scelgono un punto P appartenente alla curva di ordine n grande, in modo che $nP = O$.

Angelica Sceglie un intero positivo $n_a < n$ come chiave privata, genera una chiave pubblica $P_a = n_a B$ spedendola in chiaro a Brian.

Allo stesso modo Brian genera un punto casuale $P_b = n_b B$ che a sua volta viene spedito ad Angelica in chiaro.²

Angelica riceve il punto P_b e calcola un nuovo punto $n_a P_b = n_a n_b B = S^3$, Brian compie la stessa operazione con n_b al posto di n_a in modo da ottenere lo stesso punto S

Per trasformare S in una chiave segreta condivisa occorre trasformarlo in un numero intero, convenzionalmente si considerano gli ultimi 256 bit dell'ascissa di S , ovvero:

$$k = x_s \pmod{2^{256}} \quad (2.15)$$

2.2.2 Metodi a confronto

Quando mettendo a confronto i metodi crittografici visto fino ad ora si nota come le curve ellittiche utilizzino chiavi di dimensione notevolmente inferiore

²I Punti P_a e P_b rappresentano punti calcolati a partire da punti scelti casualmente, questi appartengono alla curva

³ S è un nuovo punto appartenente alla curva

rispetto agli altri, il che le rende molto utili per dispositivi con risorse limitate, come smartcard, dispositivi IoT...

<i>TDEA, AES</i> (bit della chiave)	<i>RSA e DH</i> (bit del modulo)	<i>ECC</i> (bit dell'ordine)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512

2.2.3 Problema del logaritmo discreto

Il problema del logaritmo discreto ellittico (ECDLP) richiede di trovare un intero m tale che

$$mP = Q \quad (2.16)$$

per dati punti $P, Q \in E(\mathbb{F}_q)$. Questo problema viene considerato computazionalmente difficile: i migliori algoritmi noti, come Pollard's ρ e Baby-step Giant-step, hanno complessità $O(\sqrt{n})$, dove n è l'ordine del punto P . Inoltre, è stato dimostrato che nessun algoritmo generico può superare questo limite [3]

2.2.4 Algoritmi di attacco

Theorem 2.2.2 (Baby-step Giant-step – Shanks). *Dato un gruppo G , siano $x, y \in G$ e n l'ordine di x . Il seguente algoritmo risolve il PLD in $O(\sqrt{n})$ operazioni e $O(\sqrt{n})$ spazio [3]:*

1. Poniamo $N = \lceil \sqrt{n} \rceil$.
2. Calcoliamo i *babystep*:

$$x, x^2, x^3, \dots, x^N. \quad (2.17)$$

3. Poniamo $z = (x^N)^{-1}$ e calcoliamo i *giantstep*:

$$yz, yz^2, yz^3, \dots, yz^N. \quad (2.18)$$

4. Se esiste un'uguaglianza $x^i = yz^j$ tra le due liste, allora $y = x^{i+jN}$; altrimenti y non è una potenza di x .

La dimostrazione si può trovare in [3].

proposition 2.2.1 (Pohlig-Hellman attack). *L'algoritmo di Pohlig-Hellman [5] riduce l'ECDLP in un gruppo di ordine n a sottoproblemi nei sottogruppi di ordine primo di $\langle P \rangle$, utilizzando il Teorema Cinese del Resto per ricostruire la soluzione. La complessità dell'algoritmo dipende dal fattore primo più grande di n : se n è B -smooth, l'attacco diventa efficiente. Per questa ragione, i parametri della curva devono essere scelti in modo che n sia divisibile da un primo grande.*

Chapter 3

Protocolli basati su Curve Ellittiche

In questa sezione si affronteranno due protocolli applicati alle curve ellittiche: *firma digitale* e *cifratura integrata*

3.1 Firma digitale

Con lo sviluppo di internet si sono venuti a creare nuovi problemi, quelli che verranno discussi in questa sezione sono:

- **Identificazione:** un sistema di elaborazione deve essere in grado di accertare l'identità di un utente che richiede di accedere ai suoi servizi.
- **Autenticazione:** il destinatario di un messaggio deve essere in grado di accertare l'identità del mittente e l'integrità del crittogramma ricevuto.

Le firme digitali sono utilizzate per rappresentare una versione digitale della firma cartacea. Queste sono rappresentate da un numero basato su un segreto noto solamente all'utente che intende firmare (chiave privata), e in aggiunta sul contenuto del documento che viene firmato. Un aspetto fondamentale delle firme digitali è che ogni firma deve essere verificabile — se dovesse sorgere una controversia, un'entità terza imparziale deve essere in grado di risolvere la questione in modo equo senza la necessità di avere accesso alla chiave privata del mittente. Queste dispute possono sorgere nei seguenti casi:

- il mittente intende negare di aver firmato un messaggio da lui effettivamente firmato.

- il destinatario sostiene di aver ricevuto un messaggio diverso da quello inviatogli dal mittente.

La proprietà di sicurezza fondamentale di uno schema di firma digitale è l'*existential unforgeability under chosen-message attack*: un attaccante in grado di ottenere la firma dell'utente A per qualunque messaggio a sua scelta non deve essere in grado di falsificare la firma di A su un nuovo messaggio. [7]

3.1.1 Generazione delle chiavi per comunicare

prerequisiti

Per assicurarsi di utilizzare una curva ellittica che non sia vulnerabile ad attacchi crittografici noti è importante tenere in considerazione questi aspetti quando durante la scelta della curva:

1. un campo finito di ordine q , dove o $q = p^1$, un numero primo dispari, oppure $q = 2^m$
2. un'indicazione FR (*field representation*) della rappresentazione usata per gli elementi di $\mathbb{F}_q$
3. due elementi a, b che definiscono la curva ellittica E definita su $\mathbb{F}_q$ (es. $y^2 = x^3 + ax + b$ nel caso di $p > 3$, e $y^2 + xy = x^3 + ax^2 + b$ in caso $p = 2$)
4. due elementi del campo x_G e y_G su $\mathbb{F}_q$ i quali definiscono un punto finito $G = (x_G, y_G)$ di ordine primo in $E(\mathbb{F}_q)$
5. l'ordine n del punto G , con $n > 2^{160}$ e $n > 4\sqrt{q}$
6. il cofattore $h = \frac{\#E(\mathbb{F}_q)}{n}$

Per una spiegazione più dettagliata della motivazione dietro alla scelta di questi requisiti consultare [7]

calcolo effettivo

Le coppie di chiavi di un dato utente A , *chiave pubblica* - *chiave privata* sono associate ad un insieme specifico dei parametri del dominio di una curva ellittica denominati $D = (q, FR, a, b, G, n, h)$, i valori che questi devono assumere

¹ p rappresenta la caratteristica del campo, importante quando si tratta campi come F_{2^m}

sono stati discussi precedentemente. È necessario che ogni utente si assicuri che i parametri del dominio siano validi prima della creazione delle chiavi [7]. Per generare una coppia di chiavi, *pubblica* - *privata*, ogni utente deve seguire tre passi:

1. Selezionare un numero casuale, o pseudo-casuale, d compreso nell'intervallo $[1, n - 1]$.
2. Calcolare $Q = dG$.
3. La chiave privata dell'utente è Q ; la chiave pubblica è d .

3.1.2 Verifica della chiave pubblica

Per validare una chiave pubblica Q ricevuta da un utente A , si possono usare uno di questi metodi:

1. A utilizza un algoritmo di validazione delle chiavi pubbliche, come 3.1.2
2. A genera Q utilizzando un sistema affidabile
3. A riceve conferma da un'entità affidabile T la quale ha validato la chiave utilizzando un algoritmo simile a 3.1.2
4. A riceve conferma da un'entità affidabile T che Q è stato generato utilizzando un sistema affidabile

Algoritmo di validazione della chiave pubblica

INPUT: Una chiave pubblica $Q = (x_Q, y_Q)$ associata a parametri di dominio validi (q, FR, a, b, G, n, h) .

OUTPUT: Accettazione o rifiuto della validità di Q .

1. Controllare che $Q \neq O$
2. Controllare che x_Q e y_Q sono elementi correttamente rappresentati su $\mathbb{F}_q$ (es., interi compresi nell'intervallo $[0, p - 1]$ nel caso $q = p$, e stringhe di bit di lunghezza m nel caso di $q = 2^m$).
3. Controllare che Q appartiene alla curva ellittica definita da a e b
4. Controllare che $nQ = O$
5. Se qualunque di questi controlli risultano falsi, allora Q non è valido; altrimenti

3.1.3 Creazione di una firma digitale

Per firmare un messaggio m , un utente A con parametri di dominio $D = (q, FR, a, b, G, n, h)$ con chiavi (d, Q) deve seguire i seguenti passaggi:

1. Selezionare un numero casuale, o pseudocasuale, k all'interno dell'intervallo $1 \leq k \leq n - 1$.
2. Calcolare $kG = (x_1, y_1)$ e convertire x_1 in un intero $\bar{x}_1$.
3. Calcolare $r = x_1 \bmod n$. Se $r = 0$ allora tornare allo step 1.
4. Calcolare $k^{-1} \bmod n$.
5. Calcolare $\text{SHA-1}(m)$ e convertire la stringa di bit in un intero e .
6. Calcolare $s = k^{-1}(e + dr) \bmod n$. Se $s = 0$ allora tornare allo step 1.
7. La firma dell'utente A per il messaggio m è (r, s) .

3.1.4 Verifica di una firma digitale

Per verificare la firma (r, s) dell'utente A per il messaggio m , l'utente B ottiene una copia autentica dei parametri di dominio di A : $D = (q, FR, a, b, n, h)$ e la chiave pubblica Q ad esso associata.

Una volta validati sia B che Q (per una dimostrazione formale consultare [7]), B deve procedere in questa maniera:

1. Verificare che r e s siano interi che appartengono all'intervallo $[1, n - 1]$.
2. Calcolare $\text{SHA-1}(m)$ e convertire la string di bit in un intero e .
3. Calcolare $w = s^{-1} \bmod n$.
4. Calcolare $w_1 = ew \bmod n$ e $w_2 = rw \bmod n$.
5. Calcolare $X = w_1G + w_2Q$.
6. Se $X = O$, allora rifiutare la firma. Altrimenti convertire la coordinata x di X in un intero $\bar{x}_1$ e calcolare $v = \bar{x}_1 \bmod n$.
7. Accettare la firma solo e soltanto se $v = r$.

Bibliography

- [1] L. Margara, *Crittografia su Curve Ellittiche*, Unibo, 2026.
- [2] A. Languasco, A. Zaccagnini, *Introduzione alla Crittografia. Algoritmi, Protocolli, Sicurezza Informatica*, Springer, 2004.
- [3] J. H. Silverman, *The Arithmetic of Elliptic Curves*, 2nd ed., Springer, 2009.
- [4] Enciclopedia Treccani, *Punto all'infinito*, [https://www.treccani.it/enciclopedia/punto-all-infinito_\(Enciclopedia-della-Matematica\)/](https://www.treccani.it/enciclopedia/punto-all-infinito_(Enciclopedia-della-Matematica)/), consultato il May 23, 2026.
- [5] Darrel Hankerson, Alfred Menezes, Scott Vanstone *Guide to Elliptic Curve Cryptography*, Springer, 2004
- [6] Alex Biryukov, *Encyclopedia of Cryptography, Security and Privacy*, Springer, 2025
- [7] Don Johnson, Alfred Menezes, Scott Vanstone *The Elliptic Curve Digital Signature Algorithm (ECDSA)*, Springer, 2001